

Arrayer

En 1-dimensionell array kan beskrivas ungefär som "en lista med plats för flera värden av en viss typ"

Syfte: att få tillgång till minne med plats för flera värden men endast behöva ett namn.

Syntax och betydelse:

Deklaration.

Form: `typ namn[storlek];` där storlek måste anges som ett konstant heltalsuttryck, och anger antalet platser som ska finnas i arrayen.

Ex: `int arr[100]; /* en array med plats för 100 int-värden */`

Ex: `char text[1000];`

Betydelse: Namnet blir i deklarationen en pekare till första elementet i listan.

Operationer på ett element i en array

Form: `namn[index]` Uttryckets värde är det värde som finns på plats *index* räknat från början av arrayen *namn* (första index är 0).

Exempel:

```
double list[20];
```

```
list[1] = 3.14;
```

```
list[0] = list[1]*2;
```

namn[index] kan jämföras med att bara ange namnet på en enskild variabel, alltså:

```
int a0, a1, a2, a3;
a2 = 42;
a1 = a2 * 13;
. . .
int b[4];
b[2] = 42;
b[1] = b[2] * 13;
```

Det främsta syftet är att kunna använda oss av arrayer i loopar, t.ex.:

```
int i, b[1000], x;
...
for (i = 0; i < 1000; i++)
    b[i] = i * x;
```

där vi hade varit chanslösa utan arrayer.

Att i en operation använda eller påverka flera element i en hel array är omöjligt, man måste alltid arbeta med ett element i taget. Arrayer kommer ändå att ge oss en kraftfull utökning av vår "verktygslåda" för programmering, då vi i en enda loop kan påverka hela arrayen.

Problemlösning (enkelt exempel)

Funktionsbeskrivning:

Användaren ska få skriva in ett (valfritt) antal flyttal, och programmet ska sedan utföra operationerna "beräkna summan av alla tal" och "kvadrera alla talen". Utskrift behövs då dessa två beräkningar är klara. Upp till 1000 flyttal skall klaras av.

Följande algoritmbeskrivning visar i detalj vad som skulle kunna lösa problemet.

Be om antalet tal, benämna detta: *n*
Sätt index (vi kallar det *i*) till 0

```
Utför följande n ggr:
    Be användaren skriva in ett tal
    Ta emot talet på plats i i en lista
    Öka i
Sätt att summan hittills (vi kallar den sum) är 0, och att i är 0
Utför följande n ggr:
    Addera plats i i listan till sum
    Öka i
Sätt i till 0
Utför följande n ggr:
    Kvadrera talet på plats i i listan
    Öka i
Sätt i till 0
Utför följande n ggr:
    Skriv ut värdet som finns på plats i
    Öka i
Skriv ut summan (sum)
```

C- kod:

```
int main(void)
{
    double lista[1000], sum;
    int i,n;
    do {
        printf("\nÄnge hur många värden du vill skriva in (max %d): ", 1000);
        scanf(" %d", &n);
    } while (n>1000);

    /* Här får användaren mata in sina värden och vi tar emot dem till lediga plat
for (i=0; i<n; i++) {
    printf("Skriv nästa värde:");
    scanf(" %lf", &lista[i]);
}

/* Summering: */
sum = 0;
for (i=0; i<n; i++)
    sum = sum + lista[i];

/* Kvadrering: */
for (i=0; i<n; i++)
    lista[i] = lista[i] * lista[i];

/* Utskrift av det beräknade: */
printf("Kvadraterna: ");
for (i=0; i<n; i++)
    printf(" %f", lista[i]);
printf("Summan blir: %f\n", sum);
return 0;
}
```

Vi spinner vidare med det här exemplet för att vid detta lämpliga tillfälle se en motivering till makron...

Vanligt är att man efter en tid behöver utöka kapaciteten för sitt program. Vi kan t.ex. låtsas att programmet ovan måste utökas till 2000 värden. Som erfaren programmerare vågar jag påstå att det är rätt hög sannolikhet att vi missar ett av de 3 ställen vi måste ändra på.

Vi simulerar nu en liten bugg:

```
int main(void)
```

```
{
    double lista[1000], sum;
    int i,n;
    do {
        printf("\nÄnge hur många värden du vill skriva in (max %d): ", 2000);
        scanf(" %d", &n);
    } while (n>2000);
```

... etc.

Både vi och användaren tror nu att programmet ovan fungerar; det påstår att 2000 är gränsen, det accepterar upp till 2000 som antal, men dess kapacitet är fortfarande 1000. **OBS!** i C finns inte någon automatisk kontroll av arraygränser. Programmet kommer alltså inte att avslutas med en varning i stil med "array index too large" då användaren skriver in det 1001:e talet. Hur ska vi undvika denna typ av misstag? - Genom att konsekvent redan från början använda makron i dessa sammanhang undviker vi denna typ av fel (som är typiskt för arrayer). Makrodefinitionen gör att det bara finns ett ställe att ändra på och om vi glömmer att ändra på detta enda så kommer det att vara uppenbart vid varje programkörning att alltihopa är ogjort:

```
#define MAXANTAL 1000
int main(void)
{
    double lista[MAXANTAL], sum;
    int i,n;
    do {
        printf("\nÄnge hur många värden du vill skriva in (max %d): ", MAXANTAL);
        scanf(" %d", &n);
    } while (n>MAXANTAL);
    ...etc
```

Funktionsanrop med arrayer som inparametrar

I deklarationen av en funktions inparameter skrivs arrayen med hakar, här tillåts alltid att storleken utelämnas på 1-dimensionella arrayer. Anledningen att detta kan tillåtas är att denna deklaration inte innebär att en ny array skapas. Det som avses med en sådan deklaration är en pekare till den array som anges i den anropande funktionen (t.ex. main). När arraynamnet tillsammans med ett index senare används i funktionen för att operera på arrayens data så utförs operationen direkt på anroparens array (dess datainnehåll kan alltså ändras). Således är arrayen som inparameter egentligen en pekare. Här är en väsentlig skillnad mot att använda arraynamnet där den deklarerats som array. I fallet med arraydeklaration är namnet till sin typ en pekare men ej en pekarvariabel. I fallet med inparameter är arraynamnet en riktig pekarvariabel som alltså kan ändra värde. Exempel som visar det senare:

```
#define MAX 10
void funk(int arrin[], int n)
{
    int farr[MAX],i;
    arrin = farr;
    /* Detta är tillåtet. Om det man tänkt sig är att kopiera värdena från den arr
       pekar på, till den plats som farr pekar på så blir man dock besviken. Detta
       inte alls vad som händer! Istället tilldelas arrin värdet av farr (som är e
       Efter tilldelningen har alltså t.ex. arrin[i] samma betydelse som farr[i] *

       . . .
}
```

```
int main(void)
{
    int arr[MAX]={1,2,3}, n=3;
    funk(arr, n);
    return 0;
}
```

Här är exempel på en normal hantering av inparameter (funk är en funktion som nollställer en int-array av längd n).

```
#define MAX 10
void funk(int arrin[], int n)
{
    int i;
    for (i=0; i<n; i++)
        arrin[i] = 0;
}

int main(void)
{
    int arr[MAX]={1,2,3}, n=3;
    funk(arr, n);
    return 0;
}
```

Eftersom array-parametrar är en pekari variabel, så kan också så kallad pekariaritmetik användas:

```
#define MAX 10
void funk(int arrin[], int n)
{
    int i;
    for (i=0; i<n; i++, arrin++)
        *arrin = 0;
}

int main(void)
{
    int arr[MAX]={1,2,3}, n=3;
    funk(arr, n);
    return 0;
}
```

....och vi kommer väl ihåg att de aktuella parametervärdena i en funktion är kopior av anroparens värden, så att i funk ändra pekarens värde med `arrin++` kan inte påverka anroparens värde.

Ojdå förresten, här dök det visst upp något nytt: Pekariaritmetik!

Man kan tydligen skriva `p++` om `p` är en pekare. Vi är vana att `++` operatoren i allmänhet betyder att öka värdet med 1. I detta fall är betydelsen något annorlunda. `p++` är visserligen ekvivalent med `p = p + 1` men detta betyder att själva adressvärdet ökas med "storleken av ett sådant element som `p` pekar på". Dvs adressökningen blir i detta fall typiskt 4, eftersom det är en pekare till int.

`p--` funkar också på analogt sätt. Allmänt, om `p` och `q` är pekare av samma typ:

`p - x` är giltigt om `x` är ett heltalsuttryck, resultatet har samma typ som `p`

`p + x` är giltigt om `x` är ett heltalsuttryck, resultatet har samma typ som `p`

`p - q` är giltigt och resulterande värde är av typen **int**

Kontentan är att `*(p + x)` är ekvivalent med `p[x]`, vilket gör att allt blir symmetriskt och vackert...

Ytterligare: $p + x$ är av samma skäl ekvivalent med $\&p[x]$, vilket ofta utnyttjas.

Anmärkning: Så länge p 's värde bara används (alltså att p inte tilldelas något) så kan p vara namnet på en array likaväl som en pekari variabel, ty arrayer är till sin typ pekare (men konstanta).

Eftersom arrayer som parametrar uppenbarligen *är* pekare, så är det kanske inte överraskande att syntaxen med $*$ kan användas i funktions huvudet:

```
#define MAX 10
void funk(int *arrin, int n)
....etc
```

...går alltså lika bra, och är faktiskt den notation man oftast ser. Om du alltså ser en prototyp till en lib-funktion (som någon annan gjort) och den har formen

```
void funk(int *s, int x)
```

så kan du inte av denna utläsa om man som första parameter ska passa en pekare till en enskild variabel eller en pekare till en array. Man måste därför också hitta en beskrivning av parametrarnas betydelse.

OBServera också att det normalt sett inte går att returnera värdet av en hel array från en funktion.

DubbelOBS: om du ger funktionen typen "pekare till arrayens elementtyp" med tanken att göra just det så är det *syntaktiskt korrekt* att göra som nedan! Det resulterar dock alltså *inte* i att arrayen returneras, istället returneras en pekare till en lokal array som är odefinierad efter returen. Pekare som returneras kan vara både bra och t.o.m. nödvändigt ibland, men dessa pekarevärden måste då peka på minne som är beständigt efter att funktionen avslutats, vanligen dynamiskt allokerat. Exempel (detta är alltså ett misstag):

```
#define MAX 10
int *funk(int arrin[], int n)
{
    int farr[MAX], i;
    for (i=0; i<n; i++)
        /* Helt OK kopiering av element från en till en annan array */
        farr[i] = arrin[i];
    return farr;    /* Förkastligt! */
}

int main(void)
{
    int arr[MAX]={1,2,3}, n=3, *arrcopy;
    arrcopy = funk(arr, n); /* Vad pekar arrcopy på efter detta anrop?
        Vad vet vi om detta minne då funktionen funk har körts färdigt? */
    for (i=0; i<n; i++)
        arrcopy[i] = arrcopy[i] * arrcopy[i];
    /*AAjj! */
    return 0;
}
```

Meningsfulla användningar returvärden av pekartyp kommer att dyka upp i förbifarten senare i kursen

Flerdimensionella arrayer

En 2-d array kan man tänka på som en tabell. Formen är i många avseenden analog med 1-d. Allmän form för n-dimensionell array:

Deklaration:

```
typ namn[storlek 1][ storlek 2]...[ storlek n];
```

I uttryck:

`namn[index 1][ index 2]...[ index n]`

Ex. vi gör en gångertabell som 2-d array (max 10):

```
#define MAX 10
int multab[MAX][MAX], i, j;
for (i=0; i<MAX; i++)
    for (j=0; j<MAX; j++)
        multab[i][j] = i * MAX + j + 1;
```

Hur ser det ut i minnet? Vi provkör med en array med olika storlek på dimensionerna:

```
int multab[3][4], i, j;
for (i=0; i<3; i++)
    for (j=0; j<4; j++)
        multab[i][j] = i * 3 + j;
```

Arrayelement Adress Värde

<code>arr[0][0]</code>	100	0
<code>arr[0][1]</code>	104	1
<code>arr[0][2]</code>	108	?
<code>arr[0][3]</code>	112	?
<code>arr[1][0]</code>	116	3
<code>arr[1][1]</code>	120	4
<code>arr[1][2]</code>	124	?
<code>arr[1][3]</code>	128	?
<code>arr[2][0]</code>	132	6
<code>arr[2][1]</code>	136	7
<code>arr[2][2]</code>	140	?
<code>arr[2][3]</code>	144	?

Betydelsen av olika uttryck med arraynamn och index:

```
int arr[3][4];
```

`arr` pekar på första elementet i den 2dimensionella arrayen, typen av uttrycket `arr` är "2d-array av `int` med plats för 4 element i dimension 2"

`arr[0]` pekar på första elementet i den 2dimensionella arrayen, typen av uttrycket `arr[0]` är "1d-array av `int`" (kan också sägas ha typen `int*`)

`arr[1]` pekar på det femte elementet i den 2dimensionella arrayen, typen av uttrycket `arr[1]` är "1d-array av `int`" (typen `int*`)

`arr[0][0]` pekar på första elementet i den 2dimensionella arrayen, typen av uttrycket `arr[0][0]` är `int`

Observera att om en funktion som tar en flerdimensionell array som inparameter så måste dennas deklaration inkludera storleken på respektive dimension. Detta skiljer från fallet med endimensionell array där man i funktionsdeklarationen inte behöver ange dimensionen (man kan i det fallet, som vi såg, t.o.m. förenkla `typ namn[]` till `typ *namn`). Skälet till att den ytterligare informationen behövs i fallet med flerdimensionella arrayer är att informationen om dimensionerna krävs för att kompilatorn ska kunna hitta rätt i arrayen (se skissen ovan: annars vore t.ex. platsen för `arr[1][3]` inte väldefinierad).

Ex. Problem: Att göra en funktion som utför skalär multiplikation på en matris (förutom denna funktion så skriver vi hela programmet så att man ser användningen av funktionen).

```
#include <stdio.h>
#include <conio.h>
```

```

#define MAX 100

void scalar_multiply(double matrix[MAX][MAX], int n, int m, double x)
{
    int i,j;
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            matrix[i][j] = matrix[i][j] * x;
        }
    }
}

int main(void)
{
    double matrix[MAX][MAX], x;
    int i,j,n,m;
    printf("Ange antal rader:");
    scanf(" %d", &n);
    printf("Ange antal kolumner:");
    scanf(" %d", &m);
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            printf("Ange rad %d kolumn %d:", i+1, j+1);
            scanf(" %lf", &matrix[i][j]);
        }
    }
    printf("Ange ett v„rde att multiplicera med:");
    scanf(" %lf", &x);
    scalar_multiply(matrix, n, m, x);
    for (i=0; i<n; i++) {
        for (j=0; j<m; j++) {
            printf(" %f", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}

```

Strängar

Arrayer kan som sagt vara av vilken typ som helst. Ibland kan man vilja ha heltalsarrayer som är av typen `char`. Det är fortfarande en heltalsarray (fast med plats endast för mycket små heltalsvärden ex:

```

char carr[100];
carr[0] = 17;
etc.

```

`char`-variabler kommer oftast till användning i samband med att man hanterar text, och man behöver ofta i samband med dessa också använda *teckenkonstanter*, t.ex. 'A' som har typen **char**.

På samma sätt används en `char`-array ofta i samband med text

Sträng är en sätt (konvention) att lagra data på ett fiffigt sätt.

En sträng är en sekvens av ASCII-koder (dvs heltal) där heltalsvärdet 0 används för att markera slutet på sekvensen (dvs en markering att inga ytterligare relevanta data finns). Syftet är att man inte behöver hålla reda på antalet relevanta tecken genom att införa en separat variabel, som är fallet för arrayer avsedda för data som representerar godtyckliga numeriska värden. Det hela möjliggörs av att det inte finns några `ascii`-koder som har just värdet 0 (t.ex. så har 0, *dvs texten 0*, `ascii`-värdet 48).

En konstant sträng har notationen, t.ex.:

"Hej"

..vilket inte bör vara någon överraskning.

Vi kan lagra strängar i variabelt minne också, t.ex. lägga in en sträng i en array av typen char.

Konstantsträngen "hej" betyder en array av 4 char-platser med värdet 104, 101, 106, 0

En char-array är alltså inte en sträng förrän vi gett den data.

```
char s[100]; /* ej sträng */
s[0] = 104;
s[1] = 101;
s[2] = 106;
/* s är fortfarande inte en sträng */
s[3] = 0;
/* Nu innehållet s en sträng (strängen hej) */
```

vi kunde också ha skrivit:

```
s[0] = 'h';
s[1] = 'e';
s[2] = 'j';
s[3] = 0;
```

Ex Initiera en char-array med strängen hej i samband med deklarationen:

```
char s[100]={104, 101, 106, 0};
```

funkar men är lite jobbigt ...

då kommer vi igen med teckenkosntanter:

```
char s[100]='h', 'e', 'j', 0;
```

och det blir lite bekvämare

men eftersom det är så vanligt med strängar finns ett ännu enklare skrivsätt för samma sak:

```
char s[100]="hej";
```

...och det var kanske det som man skulle försökt med på en gång, det är ju helt intuitivt! Så varför tjata om de andra stolliga varianterna som man aldrig använder? - Jo, för i C måste vi kunna hantera strängar på en sådan nivå att vi känner till implementationen. Vi måste alltså som C-programmerare veta implementationen av strängar.

Och glöm föralldel inte att det finns en "osynlig" nolla efter j:et! Kompilatorn hör av sig om vi skriver:

```
char s[3]="Hej";
```

men det är inte alltid som det ser ut så, t.ex.:

```
void foo(char *s, int max_space_in_s)
{
    if (max_space_in_s > 3)
        strcpy(s, "hej");
    ...
}
```

En strängkonstant är till sin typ en pekare till char, dvs char*

Därav följer att en variabel som är av typen char* kan sättas att peka på en konstantsträng. Ex:

```
char s[100]="hej";
char *p="hej";
```

Arrayen *s* *innehåller* nu strängen hej (en kopia av konstantsträngen "hej"), medan pekaren *p* pekar på konstantsträngen "hej". Båda är helt ok att använda, men endast den översta är lämplig om man senare vill ändra på innehållet i strängen. Det är alltså ok att senare ändra till *s*[0] = 'H' för att få inledande versal. Det är dock fel att göra *p*[0] = 'H'.

Det finns en speciell formatkod %s för sträng, den kan användas i scanf och printf. Ex


```
char s[100];
scanf("%s", s);
```

scanf väntar tills användaren skrivit en text och tryckt på enter. Texten (dvs ASCII-koderna för de bokstäver, eller snarare tecken, som motsvarar tangenttryckningarna) kommer att placeras i arrayen s. Dessutom kommer den att lägga till en 0-a på slutet som stoppmarkering.

```
char s[100];
printf("%s", s);
```

printf tolkar variabeln s som adressen till en char (en sträng) då %s används. Den kommer att skriva ut tecken för tecken med början på första platsen i s. Den kommer att sluta när den hittar en 0:a. Men vad händer då om användaren skriver in en 0:a i scanf-satsen?? - Jo, då kommer ascii-koden för 0 att läggas in i arrayen, och det är 48 så det blir inget problem. Ingen tangenttryckning kommer att leda till att det numeriska värdet 0 hamnar i strängen. Endast den avslutande enter-tryckningen får scanf att lägga till värdet 0. Dvs, någonstans i scanf finns satsen:

```
if (c=='\n')
    string[i] = 0;
```

Ett antal funktioner, som görs tillgängliga om man inkluderar string.h, finns att tillgå. Dessa opererar på strängar, dvs om deras uppdrag är att läsa en sträng så letar de efter slutnollan för att detektera slutet av strängen och om deras uppdrag är att skapa en sträng så lägger de till en nolla på slutet. Kolla in i boken eller on-linehjälpens så du ser några exempel på vad som finns.

Ex. Gör en funktion som beräknar längden av en sträng.

Algoritm:

```
Indata är en char*
Sätt index till 0
Upprepa följande tills slutet av strängen nåtts:
    Öka index med 1
Längden på strängen är värdet på index
```

C-kod:

```
int strlen(char *s)
{
    int i;
    for (i=0; s[i]; i++)
        ;
    return i;
}
```

s[i] är sant om värdet på platsen s[i] är skilt från 0 vilket det kommer att vara ända tills i uppnått ett sådant värde att vi nått själva strängavslutningstecknet. Man kan som villkor i for-satsen istället skriva s[i]!=0 vilket är ett mer omständligt men helt korrekt sätt att uttrycka samma sak. Dessutom kan 0 alltid uttryckas som '\0' om man så föredrar (" betyder att det är en teckenkonstant för tecknet som står mellan fnuttarna, och \ betyder att det som står omedelbart efter det inte ska tolkas som ett tecken utan som ett numeriskt värde...).

Vi kan lösa samma problem genom att använda pekararitmetik:

```
int strlen(char *s)
{
    int i;
    for (i=0; *s; i++)
```

```
        s++;  
    return i;  
}
```

Eller varför inte:

```
int strlen(char *s)  
{  
    char *p = s;  
    while (*p)  
        p++;  
    return p - s;  
    /* p pekar här på strängslutet, och differensen mellan slutet och början blir  
       längden (även utifrån ett allmänt perspektiv på hur sakers längd definieras)  
*/  
}
```

...fast vi behöver inte göra funktionen `strlen` - den är just en av de standardfunktioner som finns exporterad i `<string.h>`

Tips! En standardfunktion som är att rekommendera istället för att använda

```
scanf("%s", array);
```

är

```
gets(array);
```

Skälet är att `scanf` avbryter inläsningen då den hittar ett blanktecken (ofta vill man ha med blanktecken i godtyckliga texter, typiskt i t.ex. namn), medan `gets` inte gör det.